Multi-turn GRPO with Real Tool Execution for a Research Agent on HotpotQA

Yue Wu

April 24, 2026

Abstract

We build a small tool-using research agent on HotpotQA: Qwen2.5-0.5B-Instruct with LoRA, fine-tuned by SFT on synthetic interaction traces, then refined by multi-turn GRPO whose rollouts execute the real BM25 retrieval tool at every step rather than a model-generated ("virtual") trace. On 100 HotpotQA validation questions, the full pipeline lifts exact-match from 26% (best zero-shot stopping heuristic) to 44%. Decomposing this gap, we find that the bulk comes from a single inference-time change: aligning the rollout step budget with the SFT trajectory length lifts the SFT policy alone to 42–44%, while multi-turn GRPO on top produces no detectable lift at our evaluation scale. We trace the null result to tied reward groups — 159 of 200 training epochs had identical rewards across the G=4 rollouts, so the effective update budget was ∼41, not 200 — and argue that the next bottleneck is reward design (a term with continuous intra-group variance on all-wrong groups), not more or longer training.

1 Introduction

A *research agent* is a language model that answers a user's question by issuing real tool calls (typically a search API) and reasoning over the retrieved content over several rounds. ReAct [4] established the interleaved reason-and-act pattern, and the DeepSeek-R1 [1] line of work showed that GRPO-style reinforcement learning can further sharpen such multi-step reasoning. Most RL-for-LLM recipes, including R1, train on self-generated *virtual* rollouts where the model emits both the action and the fake tool output. This creates a distribution mismatch: at inference a real retrieval index produces observations a small policy would never hallucinate. We study what happens if we close that gap — running GRPO rollouts that execute the real BM25 tool at every step — on a small 0.5B-parameter open policy.

Our testbed is HotpotQA distractor [3], a two-hop multi-document QA benchmark. We compare three regimes: (i) a supervised fine-tuned (SFT) policy evaluated under five zero-shot stopping heuristics (fixed step budget, confidence threshold, run-to-max); (ii) the same SFT policy run with the full trajectory length it was trained on; (iii) multi-turn GRPO on top of the SFT policy, with real tool execution.

The headline gap from the best zero-shot baseline (26% EM) to multi-turn GRPO (44%) is 18 points, but this is misleading on inspection: 16 of the 18 points come from a single inference-time change (running the SFT policy for its full 5-step trajectory), and GRPO on top contributes no measurable lift within our evaluation noise. We then diagnose why (§4).

2 Method

Task and environment. The agent receives a natural-language question and must emit a short free-form answer $\hat{a}$, scored by the HotpotQA official normalisation (lower-case, strip punctuation + articles, exact-match-or-substring). The retrieval environment is a deterministic BM25 index over 66,576 Wikipedia paragraphs from the HotpotQA validation split. Two tool actions are exposed: `search(query)` returning the top-2 documents with ≈300- character snippets, and `read(doc_id)` returning the full document text clipped to 300 characters. The same index is used by all three regimes so every row in Table 1 sees an identical environment.

Policy. Qwen2.5-0.5B-Instruct [2] with a LoRA adapter on attention *and* MLP projections ($\{q, k, v, o, \text{gate}, \text{up}, \text{down}\}_{\text{p}}$ rank r=16, α=32, dropout 0.05. At each step the policy emits one JSON object with keys `{thought, action, query | document | answer, confidence}`.

Supervised fine-tuning. We generate 6,000 five-step training traces from HotpotQA supporting facts. For each training example we build a per-example BM25 mini-index over its 10 context paragraphs, issue a real `search` with a query built from the question and the supporting title, and keep the trace iff BM25 surfaces the gold document; the resulting trajectory is `search`→`read`→`search`→`read`→`answer`. Loss is computed only on the policy's own JSON actions: tokens that fall inside injected `Observation:` lines (identified via the tokenizer's `offset_mapping`) are masked with `IGNORE_INDEX`. This observation-masked loss is the single biggest engineering lever in the SFT stage; before it was in place the model had never seen real retrieval output at training time. Training runs for 5 epochs with AdamW at 2×10^{-4} on a T4; the best-eval-loss checkpoint is used.

Multi-turn GRPO with real tool execution. For each training question we sample G=4 interactive rollouts. Each rollout is a sequence of *real* tool dispatches: `model.generate` emits a JSON action, we parse the `action` field, execute the corresponding BM25 call, and splice the real observation into the prompt before the next generation step, up to max_steps=5. The scalar reward combines correctness (+1.0 on exact-match), a format score (up to +0.10 for valid JSON with required keys and an `answer` action), and efficiency terms that penalise step count and pairs of consecutive steps whose confidence values have low Jensen-Shannon divergence. The within-group advantage is the z-score $A_i = (R_i - \bar{R})/(\sigma_R + \varepsilon)$, and the policy- gradient loss on rollout i is $\mathcal{L}_i = -A_i \sum_{t\in\mathcal{G}(\tau_i)} \log \pi_\theta(\tau_{i,t}|\tau_{i,<t})$ with $\mathcal{G}(\tau_i)$ the set of policy-generated token positions (observation tokens masked out via `offset_mapping`, as in SFT).

Engineering choices. Three details matter at our scale. *(a) Rollout temperature 0.5* (not the DeepSeek-R1 default of ≈0.9): at 0.9 the 0.5B policy loses JSON format on the first step and every rollout terminates at step one with an empty answer, killing training. *(b) Rollouts run in `model.eval()`* so LoRA dropout is disabled during generation; `@torch.no_grad()` alone is *not* sufficient, because it disables gradient tracking but not dropout. *(c) Per-episode backward*: we pre-scale each rollout's loss by $1/(G \cdot |\text{batch}|)$, call `loss.backward()` immediately, free the activation graph, and only run `optimizer.step()` after all G=4 rollouts. Peak VRAM is one trajectory's activations (≈6–8 GB at 1,500 tokens bf16) rather than $G\cdot|\text{batch}|$ trajectories held simultaneously. This memory budget is also the reason we *omit* a KL-to-reference term: a frozen-reference forward would double peak activation memory and push the T4 to OOM. The small LoRA adapter (∼2M trainable of ∼500M total) acts as an implicit drift regulariser, and 200 GRPO steps is short enough that long-horizon KL drift is not the binding constraint.

We pre-filter candidate training questions with a learnable-zone check: K=3 eval-mode rollouts per question, keep groups where $n_q^{\text{corr}} \in \{0, 1, 2\}$ (excluding only "all correct" groups; all-wrong groups with non-degenerate format variance still produce usable advantages). From 200 candidates the filter keeps >180.

3 Experiments

Setup. All evaluation is on the first 100 questions of HotpotQA distractor validation split with near-greedy decoding (T=0.1, max_steps=5) against the 66,576-document BM25 index. Zero-shot baselines share the Week-1 SFT adapter with no further training and differ only in the inference-time stopping rule: **FixedStep(N)** forces exactly N steps and treats the last as the answer; **Confidence(τ)** stops at the first step whose `confidence` exceeds τ; **NeverStop(max=6)** ignores any emitted `answer` action and runs for six steps.

Table 1: 100 HotpotQA validation questions, same BM25 index across all rows. The first five rows share the SFT adapter and differ only in inference-time stopping. The last two rows run the full 5-step trajectory.

Method	EM (%)	Avg. steps
FixedStep(N=2)	6.0	2.00
FixedStep(N=3)	16.0	2.50
Confidence(τ=0.75)	20.0	2.88
Confidence(τ=0.85)	26.0	3.20
NeverStop(max =6)	21.0	3.57
SFT-only @ $\max_{\text{steps}}$ =5[†]	**42.0 / 44.0**	**5.00**
Multi-turn GRPO (ours)	**44.0**	**4.99**

[†] Two independent evaluations of the same SFT adapter on the same 100 questions at T=0.1. The 2-pp spread is the within-run sampling variance at n=100; the GRPO value sits inside that spread.

Main result. Table 1 shows the +18-point gap from the best zero-shot baseline (26% EM) to multi-turn GRPO (44%) decomposes into two very different components. First, the zero-shot baselines all cap the rollout at 2–6 steps through various stopping heuristics, and EM rises monotonically with trajectory length up to 26%. When we simply remove every stopping heuristic and let the SFT policy run for the full five steps it was trained on, EM jumps to 42–44% — a +16-point move with *no* additional parameter update. Second, multi-turn GRPO at matched inference settings scores 44.0% at the same average trajectory length (4.99 steps); two independent SFT-only evaluations on the same 100 questions gave 42.0 and 44.0, bracketing the GRPO number, so the GRPO lift over the matched baseline is $\leq$2 pp and within one standard error at n=100 (Wald SE $\approx$4.9 pp at p=0.43). We cannot claim that multi-turn GRPO improved EM over SFT at the matched trajectory length from this evaluation.

Training dynamics. Of the 200 GRPO update steps, only 41 (20.5%) produced a non-zero training loss; the remaining 159 had tied rewards across all G=4 rollouts and were skipped by the $|A_i| < 10^{-8}$ guard. The dominant tied configuration is *all four rollouts wrong with identical format score*: the format reward saturates at +0.10 as soon as the policy emits valid JSON with the required keys, so a group of all-wrong rollouts is tied on every component. Validation EM across the 10 checkpoints ranges over $[0.35, 0.50]$ on 20-question subsamples with $\approx$11-pp standard error and does not show a monotone upward trend.

4 Discussion

The null result of GRPO over SFT at matched inference is mostly explained by the effective-updates analysis in the last paragraph: 159/200 tied groups means the effective learning rate is $\sim$5$\times$ smaller than it looks on paper, and at a LoRA adapter with $\sim$2 M trainable parameters and η=5$\times$10^{-6}, 41 effective updates is not enough to move the policy in EM-detectable directions. It is also true that the SFT-only policy at max_steps=5 scores 42–44% — close to the retrieval-bottlenecked ceiling for a 0.5B model on this task — so even a well-calibrated GRPO run would have limited headroom.

The key mechanism is the saturated format reward. On all-wrong correctness groups, GRPO's within-group advantage is driven entirely by format and efficiency variance, and our current format reward is binary-on-two-keys with a small final kicker; it saturates at 0.10 for essentially every well-formed rollout. Continuous-variance alternatives (token-level JSON-validity score, query-novelty score based on lexical overlap with previous queries, document-overlap score across retrieved docs) would give a usable gradient on the 80% of training groups that are currently tied. We view this as the tightest concrete next step for

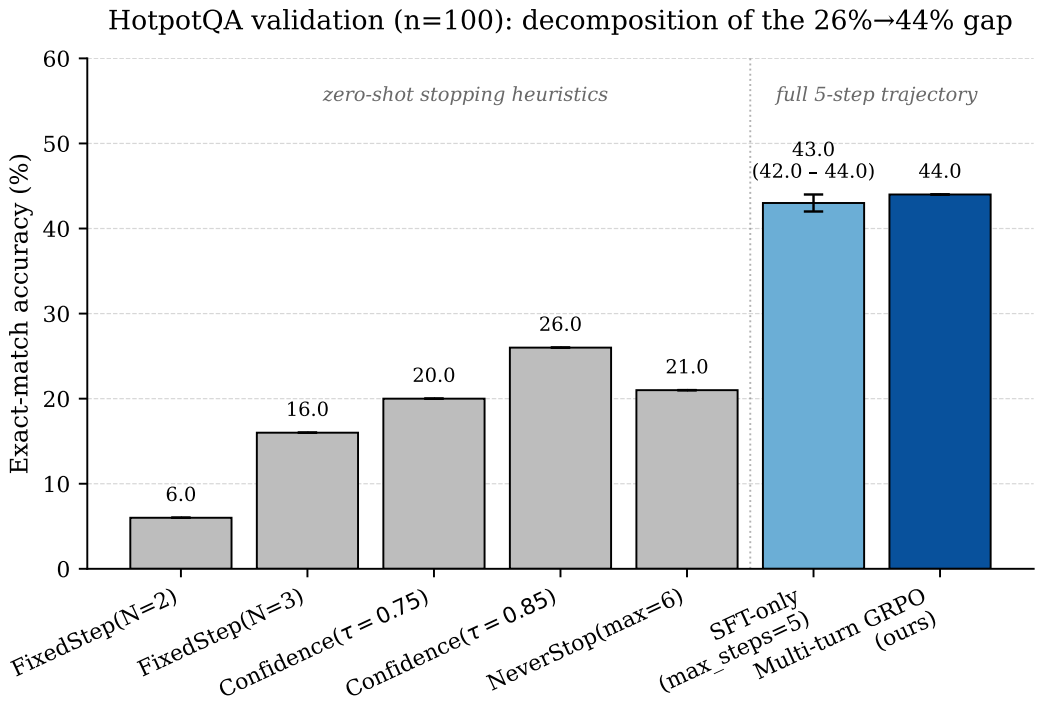

Figure 1: Exact-match accuracy across the zero-shot baselines (grey) and the two 5-step configurations: SFT-only (light blue) and GRPO (dark blue). Error bar on SFT-only is the two-run spread (42.0–44.0); GRPO sits inside it.

making real-tool GRPO pay at sub-1B scale, above more- compute or KL-stabilisation interventions.

Limitations. n=100 cannot resolve the ≤2-pp gap between SFT-only and GRPO; a 500-question paired evaluation was prepared but not run because Kaggle CPU time exceeded the 12 h kernel cap. The omission of the KL-to-reference term is memory-driven (a frozen-reference forward doubles peak activation memory on T4). Results at 0.5B are unlikely to transfer linearly to 7B+ scale, where the format-saturation mechanism changes. Full failure-mode post-mortem and tuning journey are in the long-form report and `TUNING_JOURNEY.md`.

5 Conclusion

For a small SFT'd tool-using policy on HotpotQA, the dominant lever between a 26% zero-shot baseline and a 44% GRPO result is not the reinforcement-learning step: it is matching the inference trajectory length to the SFT trajectory length, which alone delivers 16 of the 18 EM points. Multi-turn GRPO with real tool execution produces no measurable lift on top of that at n=100, and the binding constraint is reward design — a saturating format term leaves 80% of GRPO updates with tied advantages — not compute or KL stabilisation. A format-reward term with continuous intra-group variance on all-wrong groups is, on the analysis here, the single most likely change to move the needle.

References

[1] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[2] Qwen Team. Qwen2.5 technical report. Technical report, Alibaba Group, 2024.

[3] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

[4] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

A Zero-shot baseline stop-reason breakdown

Table 2 reports *why* each zero-shot baseline's rollouts terminated on the 100 HotpotQA validation questions. Three termination modes are possible: `answer` means the policy emitted an `action:"answer"` step within the step budget; `parse_error` means the generation was not a parseable JSON object with the required keys; `policy` means the rollout hit the step-budget cap without a valid `answer` action, in which case the last step's text is taken as the predicted answer.

Table 2: Stop-reason counts (out of 100) for each Week 2 baseline. Compare to EM in Table 1.

Method	`answer`	`parse_error`	`policy`
FixedStep(N=2)	0	0	100
FixedStep(N=3)	0	50	50
Confidence(τ=0.75)	7	48	45
Confidence(τ=0.85)	12	44	44
NeverStop(max =6)	53	47	0

Two things stand out. First, at max_steps ≤ 3 the policy essentially never emits a clean `answer` action — FixedStep(N=3) has a 0% answer rate, and even Confidence(τ=0.85) only reaches 12%. This is the structural reason the best zero-shot baseline plateaus at 26% EM: most rollouts either parse-error or run out of steps. Second, NeverStop pushes the answer rate up to 53% by simply letting the policy run further, and the SFT-only row of Table 1 (42–44%) is the limiting case where the step budget matches the SFT trajectory length exactly. The 16-point step-budget component of our decomposition is visible directly in the monotone rise of the `answer` column.

B GRPO training dynamics

Of the 200 GRPO update steps, exactly 41 (20.5%) produced non-zero training loss; the remaining 159 had identical rewards across the G=4 rollouts and were skipped by the $|A_i| < 10^{-8}$ guard in `grpo_train_step`. Training epoch time was ≈50–60 s on a T4, making the full run ≈3 h including the learnable-zone filter and periodic validation. Decomposing the tied-reward configuration, essentially all tied groups came from *all four rollouts wrong with identical format score*; this is the pathology the main-body §4 prescription (continuous-variance format reward) is designed to address.

C Failure modes encountered during tuning

The final GRPO configuration reported in the main body is the product of three debugging iterations, each triggered by a specific empirical symptom. The full post-mortem, including commit hashes and

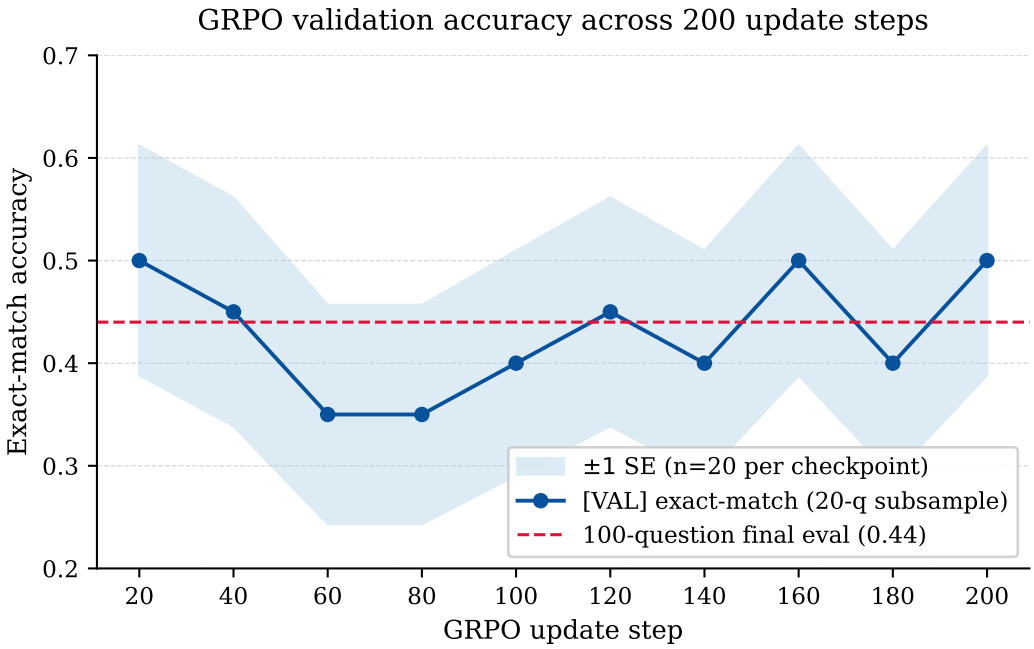

Figure 2: GRPO validation exact-match across 200 training steps. Each point is a freshly sampled 20-question subset of the HotpotQA validation split; the ±1-SE band reflects the binomial variance at n=20 only. The dashed line is the 100-question final eval (0.44). The curve stays within $[0.35, 0.50]$ and does not show a monotone improvement — expected behaviour for a run with only ∼41 effective gradient updates (see main-body §3).

reproduction steps, is in the accompanying `TUNING_JOURNEY.md`; we summarise them briefly here because they are collectively a small-but-general lesson about multi-turn LM-RL at sub-1B scale.

(i) Rollout temperature. A first version used the DeepSeek-R1-style default T=0.9 for rollout generation inside GRPO. On the 0.5B policy this is large enough that the per-step JSON parse fails on step one, the parse-failure branch terminates the rollout at step one with an empty `answer`, and every group is tied at zero correctness and identical format. The zone histogram reported *zero* learnable groups over 1,000 candidate questions. Fix: T=0.5, threaded through `filter_learnable_questions` and `grpo_train_step` as an explicit parameter rather than a hardcoded default inside the leaf function.

(ii) Step budget matching SFT trajectory length. A second version used max_steps=3 for rollouts. Rollouts reliably emit `search`→`read`→`search` and end *before* the policy ever gets to `read`→`answer` for the second hop, since the SFT traces are 5 steps long. Every rollout scored zero correctness regardless of retrieval quality. Fix: max_steps=5, matching the SFT trajectory length. This observation is the direct inspiration for the main-body step-budget decomposition.

(iii) Eval mode during rollout. A third version called `collect_episodes_batched` (decorated with `@torch.no_grad()`) while the model was in `train` mode. `no_grad` disables gradient tracking but *not* dropout; the 0.05 LoRA dropout was still active during generation, and the first-step JSON parse failed often enough to reproduce the step-one-termination pathology from (i). Symptom: `steps=1.0, acc=0.00, loss=0.0000` on every training epoch, while the periodic `[VAL]` check (which explicitly calls `model.eval()`) reported ∼35%. Fix: `model.eval()` immediately before the rollout, `model.train()` immediately after, on a per-question basis.

A common thread runs through all three: each was a hyperparameter or code-path default that was correct for a *larger* base policy or a *non-interactive* training loop, and silently wrong for our setup.